

```
"""
```

```
|| DAILY STOCK ADVISOR BOT - COMPLETE FILE ||
||
|| SETUP INSTRUCTIONS:
|| 1. Install libraries:
||     pip install anthropic finnhub-python requests matplotlib
||     pandas schedule reportlab gtts praw requests-cache numpy
||
|| 2. Fill in your API keys in the CONFIG section below
||
|| 3. Run immediately for testing:
||     python stock_advisor_complete.py --now
||
|| 4. Run on automatic schedule:
||     python stock_advisor_complete.py
||
|| FILES CREATED AUTOMATICALLY:
|| - portfolio.json      (your portfolio data)
|| - learning_log.json   (bot performance history)
|| - dca_log.json        (dollar cost averaging log)
|| - dashboard.html      (open in browser to view dashboard)
|| - stock_report_YYYYMMDD.pdf (daily PDF report)
|| - daily_briefing_YYYYMMDD.mp3 (audio summary)
||
```

```
"""
```

```
import os
import sys
import json
import time
import smtplib
import requests
import schedule
from datetime import datetime, timedelta
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import anthropic
```

```
# ||
# || CONFIG ||
```

```

# || Fill in ALL values below before running ||
# ||=====||

FINNHUB_API_KEY      = "your_finnhub_api_key_here"
ALPHAVANTAGE_API_KEY = "your_alphavantage_api_key_here"
ANTHROPIC_API_KEY     = "your_anthropic_api_key_here"

SENDER_EMAIL         = "your_gmail_here@gmail.com"
SENDER_PASSWORD       = "your_gmail_app_password_here"
RECEIVER_EMAIL        = "your_gmail_here@gmail.com"

STARTING_CAPITAL      = 100
PAUSED                = False

# Tickers you never want suggested (add any you want blocked)
BLACKLIST              = []

# Reddit API (optional - fill in when ready for Reddit sentiment)
REDDIT_CLIENT_ID       = ""
REDDIT_CLIENT_SECRET   = ""
REDDIT_USER_AGENT      = "stock-advisor"

# ||=====||
# ||                STOCK UNIVERSE                ||
# || Add or remove tickers as you like              ||
# ||=====||

STOCK_UNIVERSE = [
    # Tech / AI
    "AAPL", "MSFT", "NVDA", "AMD", "GOOGL", "META", "AMZN",
    # Small caps
    "SOUN", "BBAI", "CLOV",
    # Energy
    "XOM", "CVX", "NEE", "ENPH",
    # Healthcare
    "JNJ", "PFE", "MRNA", "UNH",
    # Finance
    "JPM", "BAC", "GS",
    # Crypto-adjacent
    "COIN", "MSTR", "RIOT",
    # ETFs / Mutual Fund proxies
    "SPY", "QQQ", "VTI", "ARKK"
]

# Remove blacklisted tickers
STOCK_UNIVERSE = [t for t in STOCK_UNIVERSE if t not in BLACKLIST]

```

```
# [ DATA FETCHING ]
# [ ]
# [ ]

import finnhub
finnhub_client = finnhub.Client(api_key=FINNHUB_API_KEY)
claude_client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

def get_stock_quote(ticker):
    try:
        quote = finnhub_client.quote(ticker)
        if quote["pc"] == 0:
            return None
        return {
            "ticker": ticker,
            "current_price": quote["c"],
            "high": quote["h"],
            "low": quote["l"],
            "open": quote["o"],
            "prev_close": quote["pc"],
            "change_percent": round(((quote["c"] - quote["pc"]) / quote["pc"]) *
        }
    except Exception as e:
        print(f" ⚠️ Error fetching {ticker}: {e}")
        return None

def get_all_quotes():
    print(" Fetching quotes...")
    quotes = []
    for ticker in STOCK_UNIVERSE:
        q = get_stock_quote(ticker)
        if q:
            quotes.append(q)
    return quotes

def get_market_news():
    try:
        news = finnhub_client.general_news("general", min_id=0)
        return [{"headline": n["headline"], "summary": n.get("summary", "")} for
    except Exception as e:
        print(f" ⚠️ Error fetching news: {e}")
        return []

def get_historical_data(ticker):
    try:
```



```

if os.path.exists("learning_log.json"):
    with open("learning_log.json", "r") as f:
        return json.load(f)
return []

def save_learning_log(log):
    with open("learning_log.json", "w") as f:
        json.dump(log, f, indent=2)

def get_consensus_analysis(quotes, news):
    """Run bull / bear / neutral analyst personas and find agreement"""
    quotes_text = "\n".join([f"{q['ticker']}: ${q['current_price']} | {q['change_"]
    news_text = "\n".join([f"- {n['headline']}" for n in news])
    personas = [
        ("Bullish Analyst", "You are an aggressive, optimistic stock analyst who
        ("Bearish Analyst", "You are a cautious analyst who focuses on downside
        ("Neutral Analyst", "You are a balanced, data-driven analyst who weighs
    ]
    all_picks = {}
    for name, desc in personas:
        try:
            msg = claude_client.messages.create(
                model="claude-opus-4-5", max_tokens=300,
                messages=[{"role": "user", "content":
                    f"{desc}\nMarket data:\n{quotes_text}\nNews:\n{news_text}\n"
                    f"Your top 5 picks as JSON only: {{\"picks\":[\"T1\", \"T2\", \"
                ]}
            )
            clean = msg.content[0].text.replace("`json", "").replace("`", "").
            result = json.loads(clean)
            for ticker in result.get("picks", []):
                all_picks[ticker] = all_picks.get(ticker, 0) + 1
        except Exception as e:
            print(f" ⚠️ Consensus error ({name}): {e}")
    consensus = [t for t, c in all_picks.items() if c >= 2]
    return {"consensus_picks": consensus, "all_votes": all_picks}

def analyze_stocks(quotes, news):
    """Main Claude analysis - generates all picks with full details"""
    learning_log = load_learning_log()
    learning_context = ""
    if learning_log:
        recent = learning_log[-5:]
        learning_context = f"\nBased on your past performance, here is what you h

    quotes_text = "\n".join([
        f"{q['ticker']}: ${q['current_price']} | Change: {q['change_percent']}% |

```

```

        for q in quotes
    ])
    news_text = "\n".join([f"- {n['headline']}" for n in news])

    prompt = f"""
You are an expert stock market analyst and personal investment advisor.
Today is {datetime.now().strftime('%A, %B %d, %Y')}.
{learning_context}

Market data:
{quotes_text}

News:
{news_text}

Provide picks in these 7 categories:
1. Conservative pick (stable, low risk)
2. Aggressive pick (high risk, high reward)
3. Mutual Fund / ETF pick
4. Small Cap pick
5. Undervalued pick
6. Momentum pick
7. Contrarian pick (goes against current market trend)

Also include a Watchlist of 2-3 stocks not quite ready to buy.

For EACH pick provide:
- ticker, category, risk_score (1-10), confidence (High/Medium/Low)
- best_time_to_buy, stop_loss (price), sleep_well_score (1-10, how safe overnight)
- explanation (2-3 sentences, beginner-friendly plain English)
- news_sentiment (how news affects this pick)
- insider_activity (any notable executive buying/selling)
- analyst_target (Wall Street price target if known, else "Not available")
- short_interest (brief description, e.g. "Low - 2% of float")

Also provide:
- market_mood: 2-3 sentence plain English market summary
- earnings_events: upcoming earnings or Fed events to watch
- sector_heatmap: {"hot": [...], "cooling": [...]}

Respond with VALID JSON ONLY. No extra text, no markdown fences.
{{
    "market_mood": "string",
    "picks": [
        {{
            "category": "string", "ticker": "string", "risk_score": 0,
            "confidence": "string", "best_time_to_buy": "string", "stop_loss": 0.0,

```

```

        "sleep_well_score": 0, "explanation": "string", "news_sentiment": "string",
        "insider_activity": "string", "analyst_target": "string", "short_interest":
    }}
],
"watchlist": ["T1","T2","T3"],
"earnings_events": "string",
"sector_heatmap": {"hot": [], "cooling": []}}
}}
"""

    msg = claude_client.messages.create(
        model="claude-opus-4-5", max_tokens=2500,
        messages=[{"role": "user", "content": prompt}]
    )
    clean = msg.content[0].text.replace("```json","").replace("```","").strip()
    return json.loads(clean)

def predict_earnings_surprise(ticker, historical_data):
    try:
        if not historical_data:
            return {"ticker": ticker, "prediction": "Insufficient data", "confidence": 0}
        dates = list(historical_data.keys())[:30]
        prices = [float(historical_data[d]["4. close"]) for d in dates]
        avg_change = sum(prices[i] - prices[i+1] for i in range(len(prices)-1)) /
        msg = claude_client.messages.create(
            model="claude-opus-4-5", max_tokens=200,
            messages=[{"role": "user", "content":
                f"Based on recent price trend for {ticker} (avg daily change: ${a
                f"predict if next earnings will beat or miss. JSON only: "
                f'{{\\"prediction\\":\\"Beat/Miss/In-line\\",\\"confidence\\":\\"High/Me
            }}]
        )
        clean = msg.content[0].text.replace("```json","").replace("```","").strip()
        return json.loads(clean)
    except Exception as e:
        return {"ticker": ticker, "prediction": "Unavailable", "confidence": "Low"}

def get_reddit_sentiment(ticker):
    """Optional - requires REDDIT_CLIENT_ID and REDDIT_CLIENT_SECRET in config"""
    if not REDDIT_CLIENT_ID or not REDDIT_CLIENT_SECRET:
        return {"ticker": ticker, "sentiment": "Neutral", "mentions": 0, "summary": ""}
    try:
        import praw
        reddit = praw.Reddit(
            client_id=REDDIT_CLIENT_ID,
            client_secret=REDDIT_CLIENT_SECRET,
            user_agent=REDDIT_USER_AGENT
        )

```

```
posts = list(reddit.subreddit("wallstreetbets+stocks+investing").search(ticker)
if not posts:
    return {"ticker": ticker, "sentiment": "Neutral", "mentions": 0, "summary": ""}
post_texts = "\n".join([f>Title: {p.title} | Score: {p.score}" for p in posts])
msg = claude_client.messages.create(
    model="claude-opus-4-5", max_tokens=150,
    messages=[{"role": "user", "content": f"Sentiment of these Reddit posts about {ticker}. JSON only: [{"sentiment": "Bullish/Bearish/Neutral"}, {"summary": "one sentence"}]}]
)
clean = msg.content[0].text.replace("`json","").replace("`","").strip()
return json.loads(clean)
except Exception as e:
    return {"ticker": ticker, "sentiment": "Neutral", "mentions": 0, "summary": ""}

def get_whale_activity(ticker):
    """Check SEC EDGAR for recent institutional filings (free)"""
    try:
        headers = {"User-Agent": "stock-advisor research@example.com"}
        start_dt = (datetime.now() - timedelta(days=30)).strftime("%Y-%m-%d")
        end_dt = datetime.now().strftime("%Y-%m-%d")
        url = (f"https://efits.sec.gov/LATEST/search-index?q=%22{ticker}%22"&dateRange=custom&startdt={start_dt}&enddt={end_dt}&forms=1)
        r = requests.get(url, headers=headers, timeout=10)
        data = r.json()
        hits = data.get("hits", {}).get("hits", [])
        if not hits:
            return {"ticker": ticker, "activity": "No recent institutional filing"}
        filings = [{"filer": h.get("_source",{}).get("entity_name","Unknown"),
                    "date": h.get("_source",{}).get("file_date","Unknown")} for h in hits]
        return {"ticker": ticker, "activity": f"{len(hits)} institutional filings",
                "filing_count": len(hits), "recent_filers": filings}
    except Exception as e:
        return {"ticker": ticker, "activity": f"Whale data unavailable: {e}", "filing_count": 0}

# ===== PORTFOLIO TRACKER =====
#
#
PORTFOLIO_FILE = "portfolio.json"
DCA_FILE = "dca_log.json"

def load_portfolio():
    if os.path.exists(PORTFOLIO_FILE):
        with open(PORTFOLIO_FILE, "r") as f:
```



```

        return json.load(f)
    return {"cash": STARTING_CAPITAL, "positions": {}, "total_invested": 0, "hist

def save_portfolio(portfolio):
    with open(PORTFOLIO_FILE, "w") as f:
        json.dump(portfolio, f, indent=2)

def add_position(ticker, shares, price):
    portfolio = load_portfolio()
    cost = shares * price
    if cost > portfolio["cash"]:
        print(f"❌ Insufficient funds. Available: ${portfolio['cash']:.2f}, Neede
        return False
    if ticker in portfolio["positions"]:
        ex      = portfolio["positions"][ticker]
        total   = ex["shares"] + shares
        avg     = ((ex["shares"] * ex["avg_price"]) + cost) / total
        portfolio["positions"][ticker] = {"shares": total, "avg_price": avg, "dat
    else:
        portfolio["positions"][ticker] = {"shares": shares, "avg_price": price, "
    portfolio["cash"] -= cost
    portfolio["total_invested"] += cost
    portfolio["history"].append({"action": "BUY", "ticker": ticker, "shares": shares, "
    save_portfolio(portfolio)
    print(f"✅ Bought {shares} shares of {ticker} at ${price}")
    return True

def remove_position(ticker, shares, price):
    portfolio = load_portfolio()
    if ticker not in portfolio["positions"]:
        print(f"❌ {ticker} not in portfolio")
        return False
    proceeds = shares * price
    portfolio["cash"] += proceeds
    if shares >= portfolio["positions"][ticker]["shares"]:
        del portfolio["positions"][ticker]
    else:
        portfolio["positions"][ticker]["shares"] -= shares
    portfolio["history"].append({"action": "SELL", "ticker": ticker, "shares": shares,
    save_portfolio(portfolio)
    print(f"✅ Sold {shares} shares of {ticker} at ${price}")
    return True

def get_portfolio_value(quotes):
    portfolio = load_portfolio()
    total_value = portfolio["cash"]
    positions = []

```

```

for ticker, pos in portfolio["positions"].items():
    q = next((x for x in quotes if x["ticker"] == ticker), None)
    if q:
        cur_val      = pos["shares"] * q["current_price"]
        cost_basis   = pos["shares"] * pos["avg_price"]
        gl           = cur_val - cost_basis
        gl_pct       = (gl / cost_basis) * 100
        total_value += cur_val
        positions.append({
            "ticker": ticker, "shares": pos["shares"],
            "avg_price": pos["avg_price"], "current_price": q["current_price"]
            "current_value": round(cur_val,2), "gain_loss": round(gl,2),
            "gain_loss_pct": round(gl_pct,2)
        })
return {
    "total_value":      round(total_value, 2),
    "cash":            round(portfolio["cash"], 2),
    "starting_capital": STARTING_CAPITAL,
    "total_return":     round(total_value - STARTING_CAPITAL, 2),
    "total_return_pct": round(((total_value - STARTING_CAPITAL) / STARTING_C
    "positions":        positions
}

def check_volatility_alerts(quotes, threshold=5.0):
    alerts = []
    for q in quotes:
        if abs(q["change_percent"]) >= threshold:
            alerts.append({
                "ticker": q["ticker"], "change_percent": q["change_percent"],
                "direction": "📈 UP" if q["change_percent"] > 0 else "📉 DOWN",
                "current_price": q["current_price"]
            })
    return alerts

def check_tax_loss_harvesting(quotes):
    portfolio = load_portfolio()
    candidates = []
    for ticker, pos in portfolio["positions"].items():
        q = next((x for x in quotes if x["ticker"] == ticker), None)
        if q:
            cost_basis = pos["shares"] * pos["avg_price"]
            cur_val    = pos["shares"] * q["current_price"]
            loss       = cur_val - cost_basis
            if loss < 0 and abs(loss / cost_basis) > 0.10:
                candidates.append({
                    "ticker": ticker, "shares": pos["shares"],
                    "avg_price": pos["avg_price"], "current_price": q["current_pr

```

```

        "unrealized_loss": round(loss,2),
        "loss_pct": round((loss / cost_basis)*100, 2)
    })
    return candidates

def log_dca_purchase(ticker, amount):
    log = []
    if os.path.exists(DCA_FILE):
        with open(DCA_FILE,"r") as f:
            log = json.load(f)
    log.append({"ticker": ticker, "amount": amount, "date": datetime.now().strftime("%Y-%m-%d %H:%M:%S")})
    with open(DCA_FILE,"w") as f:
        json.dump(log, f, indent=2)

def get_dca_summary():
    if not os.path.exists(DCA_FILE):
        return {}
    with open(DCA_FILE,"r") as f:
        log = json.load(f)
    summary = {}
    for e in log:
        t = e["ticker"]
        if t not in summary:
            summary[t] = {"total_invested": 0, "purchases": 0}
        summary[t]["total_invested"] += e["amount"]
        summary[t]["purchases"] += 1
    return summary

def what_if_simulator():
    log = load_learning_log()
    if not log:
        return {"message": "No suggestion history yet. Check back after a few day"}
    simulated = STARTING_CAPITAL
    trades = []
    for entry in log:
        for pick in entry.get("picks_performance", []):
            if pick.get("actual_return"):
                trade_val = simulated * 0.1
                gain = trade_val * (pick["actual_return"] / 100)
                simulated += gain
                trades.append({"ticker": pick.get("ticker","N/A"), "return": pick["actual_return"]})
    return {
        "starting_capital": STARTING_CAPITAL,
        "simulated_value": round(simulated, 2),
        "simulated_return_pct": round(((simulated - STARTING_CAPITAL) / STARTING_CAPITAL) * 100, 2),
        "total_trades": len(trades),
        "trades": trades[-10:]
    }

```

```

    }

def get_last_week_winner():
    log = load_learning_log()
    if not log:
        return None
    for entry in reversed(log):
        if entry.get("picks_performance"):
            best = max(entry["picks_performance"], key=lambda x: x.get("actual_re
            return best
    return None

def get_high_conviction_picks():
    log = load_learning_log()
    ticker_count = {}
    for entry in log:
        for t in entry.get("tickers_suggested", []):
            ticker_count[t] = ticker_count.get(t, 0) + 1
    return [t for t, c in ticker_count.items() if c >= 3]

def get_portfolio_grade(quotes):
    if not quotes:
        return "N/A"
    positive = sum(1 for q in quotes if q["change_percent"] > 0)
    ratio = positive / len(quotes)
    if ratio >= 0.8: return "A"
    if ratio >= 0.6: return "B"
    if ratio >= 0.4: return "C"
    if ratio >= 0.2: return "D"
    return "F"

def get_fear_greed(quotes):
    if not quotes:
        return "Neutral 😐", "#e67e22"
    avg = sum(q["change_percent"] for q in quotes) / len(quotes)
    if avg > 2: return "Extreme Greed 😄", "#27ae60"
    if avg > 0.5: return "Greed 😊", "#2ecc71"
    if avg > -0.5: return "Neutral 😐", "#e67e22"
    if avg > -2: return "Fear 😟", "#e74c3c"
    return "Extreme Fear 😱", "#c0392b"

def save_today_to_learning_log(analysis, quotes):
    log = load_learning_log()
    tickers = [p["ticker"] for p in analysis.get("picks", [])]
    entry = {
        "date": datetime.now().strftime("%Y-%m-%d"),
        "tickers_suggested": tickers,

```

```

    "market_mood":      analysis.get("market_mood", ""),
    "sector_heatmap":   analysis.get("sector_heatmap", {}),
    "picks_performance": []
}
log.append(entry)
save_learning_log(log)

def update_performance_in_log(quotes):
    log = load_learning_log()
    yesterday = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
    for entry in log:
        if entry.get("date") == yesterday and not entry.get("picks_performance"):
            perf = []
            for ticker in entry.get("tickers_suggested", []):
                q = next((x for x in quotes if x["ticker"] == ticker), None)
                if q:
                    perf.append({"ticker": ticker, "actual_return": q["change_per
            entry["picks_performance"] = perf
    save_learning_log(log)

# =====
# EMAIL BUILDER
# =====

def build_daily_email(analysis, quotes, portfolio_value, consensus, commodity_dat
    today = datetime.now().strftime('%A, %B %d, %Y')
    last_winner = get_last_week_winner()
    high_conv = get_high_conviction_picks()
    grade = get_portfolio_grade(quotes)
    fg_label, fg_color = get_fear_greed(quotes)
    total_ret = portfolio_value.get("total_return_pct", 0)
    ret_color = "#27ae60" if total_ret >= 0 else "#e74c3c"
    ret_arrow = "▲" if total_ret >= 0 else "▼"

    html = f"""
<html>
<body style="font-family:Arial,sans-serif;max-width:800px;margin:0 auto;padding:2
<h1 style="color:#2c3e50;"><img alt="Daily Stock Advisor logo" data-bbox="339 815 361 831"/> Daily Stock Advisor – {today}</h1>

<!-- Stats row -->
<div style="display:flex;gap:12px;margin-bottom:20px;flex-wrap:wrap;">
    <div style="flex:1;min-width:150px;background:#fff;padding:15px;border-radius:8
        <div style="color:#999;font-size:12px;">Portfolio Value</div>
        <div style="font-size:26px;font-weight:bold;color:#3498db;">${portfolio_value
    </div>
    <div style="flex:1;min-width:150px;background:#fff;padding:15px;border-radius:8

```

```

        <div style="color:#999;font-size:12px;">Total Return</div>
        <div style="font-size:26px;font-weight:bold;color:{ret_color};">{ret_arrow} {
</div>
<div style="flex:1;min-width:150px;background:#fff;padding:15px;border-radius:8
    <div style="color:#999;font-size:12px;">Market Sentiment</div>
    <div style="font-size:18px;font-weight:bold;color:{fg_color};">{fg_label}</di
</div>
<div style="flex:1;min-width:150px;background:#fff;padding:15px;border-radius:8
    <div style="color:#999;font-size:12px;">Portfolio Grade</div>
    <div style="font-size:36px;font-weight:bold;color:#3498db;">{grade}</div>
</div>
</div>
"""

    # Last week's winner
    if last_winner:
        html += f"""
<div style="background:#d5f5e3;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#27ae60;margin:0 0 8px;">🏆 Last Week's Top Performer</h2>
    <p style="margin:0;"><strong>{last_winner.get('ticker','N/A')}</strong> returne
</div>"""

    # High conviction
    if high_conv:
        html += f"""
<div style="background:#fef9e7;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#f39c12;margin:0 0 8px;">🎯 High Conviction Picks (Suggested 3
    <p style="margin:0;"><strong>{' '.join(high_conv)}</strong></p>
</div>"""

    # Consensus
    if consensus and consensus.get("consensus_picks"):
        html += f"""
<div style="background:#eaf4fb;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#2980b9;margin:0 0 8px;">👥 Multi-Model Consensus</h2>
    <p style="margin:0;">All 3 analyst personas agree on: <strong>{' '.join(consen
</div>"""

    # Commodity correlation
    if commodity_data.get("commodity_data"):
        cd = commodity_data["commodity_data"]
        cd_rows = "".join([f"<li><strong>{k}</strong> ${v['price']}" if v['
        html += f"""
<div style="background:#f5eef8;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#8e44ad;margin:0 0 8px;">📊 Commodity Correlation</h2>
    <ul style="margin:5px 0 0;">{cd_rows}</ul>
    <p style="margin:5px 0 0;">{commodity_data.get('insight','')}</p>

```

```
</div>""
```

```
    # Market mood
    html += f"""
<div style="background:#fff;padding:15px;border-radius:8px;margin-bottom:20px;bor
<h2 style="color:#2c3e50;margin:0 0 8px;">🌐 Market Mood</h2>
<p style="margin:0;font-size:15px;">{analysis.get('market_mood','N/A')}</p>
</div>
<h2 style="color:#2c3e50;">📈 Today's Picks</h2>"""
```

```
category_icons = {
    "Conservative pick": "🛡️", "Aggressive pick": "🚀",
    "Mutual Fund / ETF pick": "📦", "Small Cap pick": "🔍",
    "Undervalued pick": "💎", "Momentum pick": "⚡", "Contrarian pick": "🔄"
}

for pick in analysis.get("picks", []):
    icon = category_icons.get(pick["category"], "📌")
    rc = "#27ae60" if pick["risk_score"] <= 3 else "#e67e22" if pick["
    cc = "#27ae60" if pick["confidence"] == "High" else "#e67e22" if p
    tq = next((q for q in quotes if q["ticker"] == pick["ticker"]), No
    arrow_str = ""
    if tq:
        a_clr = "#27ae60" if tq["change_percent"] > 0 else "#e74c3c"
        a_sym = "▲" if tq["change_percent"] > 0 else "▼"
        arrow_str = f'<span style="color:{a_clr};font-size:14px;">{a_sym} {ab

    html += f"""
<div style="background:#fff;border:1px solid #ddd;border-radius:8px;padding:15px;
<h3 style="color:#2c3e50;margin:0 0 10px;">{icon} {pick['category']} - <span st
<table style="width:100%;border-collapse:collapse;font-size:13px;">
<tr>
<td style="padding:4px 8px;"><strong>Risk:</strong> <span style="color:{rc}
<td style="padding:4px 8px;"><strong>Confidence:</strong> <span style="colo
<td style="padding:4px 8px;"><strong>Sleep Well:</strong> {pick.get('sleep_
</tr>
<tr>
<td style="padding:4px 8px;"><strong>Best Time:</strong> {pick['best_time_t
<td style="padding:4px 8px;"><strong>Stop-Loss:</strong> ${pick['stop_loss'
<td style="padding:4px 8px;"><strong>Short Interest:</strong> {pick.get('sh
</tr>
<tr>
<td colspan="3" style="padding:4px 8px;"><strong>Analyst Target:</strong> {
</tr>
</table>
<p style="margin:10px 0 5px;"><strong>📝 Analysis:</strong> {pick['explanation'
<p style="margin:5px 0;"><strong>📰 News & Sentiment:</strong> {pick['news_sent
```

```

    <p style="margin:5px 0;"><strong> Insider Activity:</strong> {pick.get('insid
</div>""

    # Watchlist
    html += f""
<div style="background:#fff3cd;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#2c3e50;margin:0 0 8px;"><img alt="👁️" data-bbox="540 162 560 176"/> Watchlist</h2>
    <p style="margin:0;">{' '.join(analysis.get('watchlist', []))}</p>
</div>""

    # Sector heatmap
    hm = analysis.get("sector_heatmap", {})
    html += f""
<div style="background:#fff;padding:15px;border-radius:8px;margin-bottom:20px;bor
    <h2 style="color:#2c3e50;margin:0 0 8px;"><img alt="🌍" data-bbox="540 315 560 329"/> Sector Heat Map</h2>
    <p style="margin:4px 0;"><strong style="color:#e74c3c;"><img alt="🔥" data-bbox="675 335 695 349"/> Hot:</strong> {' '.
    <p style="margin:4px 0;"><strong style="color:#3498db;"><img alt="❄️" data-bbox="675 355 695 369"/> Cooling:</strong> {'
</div>""

    # Earnings & events
    html += f""
<div style="background:#fff;padding:15px;border-radius:8px;margin-bottom:20px;bor
    <h2 style="color:#2c3e50;margin:0 0 8px;"><img alt="📅" data-bbox="540 468 560 482"/> Earnings & Events This Week</h2>
    <p style="margin:0;">{analysis.get('earnings_events', 'None reported.')}</p>
</div>""

    # What-if simulator
    html += f""
<div style="background:#eaf4fb;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#2980b9;margin:0 0 8px;"><img alt="💡" data-bbox="540 602 560 616"/> What-If Simulator</h2>
    <p style="margin:0;">If you had followed every suggestion since day one, your $
</div>""

    # Tax loss harvesting
    if harvest_candidates:
        rows = "".join([f"<li><strong>{c['ticker']}</strong>: {c['loss_pct']}% lo
        html += f""
<div style="background:#fde8e8;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#e74c3c;margin:0 0 8px;"><img alt="👛" data-bbox="540 775 560 789"/> Tax Loss Harvesting Opportunities<
    <ul style="margin:0;">{rows}</ul>
</div>""

    # DCA summary
    if dca_summary:
        rows = "".join([f"<li><strong>{t}</strong> ${v['total_invested']} over {
        html += f""
<div style="background:#f0f4f8;padding:15px;border-radius:8px;margin-bottom:20px;

```



```

    <h2 style="color:#2c3e50;margin:0 0 8px;"><img alt="Bar chart icon" data-bbox="538 48 558 63"/> Dollar Cost Averaging Summary</h2>
    <ul style="margin:0;">{rows}</ul>
</div>""

    html += f"""
<hr style="border:1px solid #ddd;">
<p style="color:#999;font-size:11px;"><img alt="Warning icon" data-bbox="485 163 505 178"/> This email is generated by an AI model f
</body>
</html>""

    return html

def build_weekly_email(quotes, portfolio_value, what_if):
    log = load_learning_log()
    week_ago = (datetime.now() - timedelta(days=7)).strftime("%Y-%m-%d")
    weekly = [e for e in log if e.get("date", "") >= week_ago]
    all_t = []
    for e in weekly:
        all_t.extend(e.get("tickers_suggested", []))
    counts = {}
    for t in all_t:
        counts[t] = counts.get(t, 0) + 1
    top5 = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:5]
    week_start = (datetime.now() - timedelta(days=7)).strftime('%B %d')
    today = datetime.now().strftime('%B %d, %Y')
    total_ret = portfolio_value.get("total_return_pct", 0)
    ret_color = "#27ae60" if total_ret >= 0 else "#e74c3c"
    ret_arrow = "▲" if total_ret >= 0 else "▼"

    rows = "".join([f"<li><strong>{t}</strong> - suggested {c} time(s)</li>" for
    html = f"""
<html>
<body style="font-family:Arial,sans-serif;max-width:800px;margin:0 auto;padding:2
<h1 style="color:#2c3e50;"><img alt="Bar chart icon" data-bbox="382 682 402 697"/> Weekly Performance Report - {week_start} to {today}

<div style="background:#fff;padding:15px;border-radius:8px;margin-bottom:20px;bor
    <h2 style="color:#2c3e50;"><img alt="Briefcase icon" data-bbox="398 738 418 753"/> Portfolio Summary</h2>
    <p><strong>Starting Capital:</strong> ${STARTING_CAPITAL}</p>
    <p><strong>Current Value:</strong> ${portfolio_value.get('total_value', 'N/A')}<
    <p><strong>Total Return:</strong> <span style="color:{ret_color};">{ret_arrow}</p>
    <p><strong>Cash Available:</strong> ${portfolio_value.get('cash', 'N/A')}</p>
</div>

<div style="background:#eaf4fb;padding:15px;border-radius:8px;margin-bottom:20px;
    <h2 style="color:#2980b9;"><img alt="Lightbulb icon" data-bbox="398 888 418 903"/> What-If Simulator</h2>
    <p>Following every suggestion since day one: ${STARTING_CAPITAL} → <strong>${wh
</div>

```

```

<div style="background:#fff;padding:15px;border-radius:8px;margin-bottom:20px;bor
  <h2 style="color:#2c3e50;">🔥 Most Suggested This Week</h2>
  <ul>{rows}</ul>
</div>

```

```

<div style="background:#d5f5e3;padding:15px;border-radius:8px;margin-bottom:20px;
  <h2 style="color:#27ae60;">🧠 Learning Log</h2>
  <p>The bot analyzed <strong>{len(weekly)}</strong> trading days this week. Tota
</div>

```

```

<hr>
<p style="color:#999;font-size:11px;">⚠️ Not financial advice. For educational pu
</body>
</html>"""
    return html

```

```

# ┌────────────────────────────────────────────────────────────────────────────────┐
# │                                     EMAIL SENDER                             │
# └────────────────────────────────────────────────────────────────────────────────┘

```

```

def send_email(subject, html_content, attachments=None):
    try:
        msg = MIMEMultipart('mixed')
        msg['Subject'] = subject
        msg['From'] = SENDER_EMAIL
        msg['To'] = RECEIVER_EMAIL
        msg.attach(MIMEText(html_content, 'html'))

        if attachments:
            for filepath in attachments:
                if filepath and os.path.exists(filepath):
                    with open(filepath, "rb") as f:
                        part = MIMEBase('application', 'octet-stream')
                        part.set_payload(f.read())
                        encoders.encode_base64(part)
                        part.add_header('Content-Disposition', f'attachment; filename
msg.attach(part)

        with smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:
            server.login(SENDER_EMAIL, SENDER_PASSWORD)
            server.sendmail(SENDER_EMAIL, RECEIVER_EMAIL, msg.as_string())
        print(" ✅ Email sent!")
        return True
    except Exception as e:
        print(f" ❌ Email error: {e}")

```



```

)))
story += [t, Spacer(1,0.2*inch)]

if consensus and consensus.get("consensus_picks"):
    story += [
        Paragraph("🎯 Multi-Model Consensus", head_s),
        Paragraph(f"All analysts agree: {'', '.join(consensus['consensus_p
        Spacer(1,0.15*inch)
    ]

story += [Paragraph("Market Mood", head_s), Paragraph(analysis.get('marke
story.append(Paragraph("Today's Stock Picks", head_s))

for pick in analysis.get("picks", []):
    story.append(Paragraph(f"<b>{pick['category']} - {pick['ticker']}</b>
    pd = [
        ["Risk Score",f"{pick['risk_score']}/10","Confidence",pick['confi
        ["Best Time",pick['best_time_to_buy',"Stop-Loss",f"${pick['stop_
    ]
    pt = Table(pd, colWidths=[1.5*inch]*4)
    pt.setStyle(TableStyle([
        ('BACKGROUND', (0,0), (0,-1),colors.HexColor('#f0f4f8')),
        ('BACKGROUND', (2,0), (2,-1),colors.HexColor('#f0f4f8')),
        ('GRID', (0,0), (-1,-1),0.5,colors.grey), ('PADDING', (0,0), (-1,-1),6
    ]))
    story += [pt, Paragraph(pick['explanation'], body_s), Spacer(1,0.1*in

story += [
    HRFlowable(width="100%", thickness=1, color=colors.HexColor('#3498db'
    Spacer(1,0.1*inch),
    Paragraph("💡 What-If Simulator", head_s),
    Paragraph(f"Following every suggestion: ${STARTING_CAPITAL} → ${what_
    Spacer(1,0.15*inch),
    Paragraph("🗺️ Sector Heat Map", head_s),
    Paragraph(f"🔥 Hot: {'', '.join(analysis.get('sector_heatmap',{}).get(
    Paragraph(f"❄️ Cooling: {'', '.join(analysis.get('sector_heatmap',{}).
    Spacer(1,0.15*inch),
    HRFlowable(width="100%", thickness=1, color=colors.grey),
    Spacer(1,0.1*inch),
    Paragraph("⚠️ This report is for educational purposes only and is not
]
doc.build(story)
print(f"✅ PDF generated: {filename}")
return filename
except ImportError:
    print("⚠️ reportlab not installed - skipping PDF. Run: pip install rep
    return None

```



```

        rc = "#27ae60" if pick['risk_score'] <= 3 else "#e67e22" if pick['ris
picks_html += f"<tr><td><a href='https://finance.yahoo.com/quote/{pic

consensus_html = ""
if consensus and consensus.get("consensus_picks"):
    links = "", ".join([f"<a href='https://finance.yahoo.com/quote/{t}' ta
    consensus_html = f"<p>👁️ <strong>All analysts agree on:</strong> {lin

watchlist_links = "", ".join([f"<a href='https://finance.yahoo.com/quote/{
fg_label, fg_color = get_fear_greed(quotes)
hm = analysis.get("sector_heatmap", {})

html = f'<!DOCTYPE html>

<html>
<head>
    <title>Stock Advisor Dashboard</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1.0">
    <meta http-equiv="refresh" content="300">
    <style>
        *{{box-sizing:border-box;margin:0;padding:0;}}
        body{{font-family:Arial,sans-serif;background:#0d1117;color:#e6edf3;padding:2
        h1{{color:#58a6ff;margin-bottom:4px;font-size:22px;}}
        h2{{color:#58a6ff;margin-bottom:10px;font-size:15px;}}
        .sub{{color:#8b949e;font-size:12px;margin-bottom:20px;}}
        .grid{{display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));
        .card{{background:#161b22;border:1px solid #30363d;border-radius:8px;padding:
        .card .val{{font-size:26px;font-weight:bold;margin:6px 0;}}
        .card .lbl{{color:#8b949e;font-size:12px;}}
        .sec{{background:#161b22;border:1px solid #30363d;border-radius:8px;padding:1
        table{{width:100%;border-collapse:collapse;}}
        th{{background:#21262d;padding:8px;text-align:left;font-size:12px;color:#58a6
        td{{padding:8px;border-bottom:1px solid #21262d;font-size:12px;}}
        tr:~tr td{{background:#1f2937;}}
        a{{text-decoration:none;}}
        .hot{{color:#f85149;}} .cool{{color:#58a6ff;}}
        .footer{{color:#484f58;font-size:11px;text-align:center;margin-top:20px;}}
        .mood{{background:#21262d;padding:12px;border-radius:6px;line-height:1.6;font
    </style>
</head>
<body>
    <h1>📊 Stock Advisor Dashboard</h1>
    <p class="sub">Last updated: {today} — Auto-refreshes every 5 minutes</p>

    <div class="grid">
        <div class="card"><div class="lbl">Portfolio Value</div><div class="val" styl
        <div class="card"><div class="lbl">Total Return</div><div class="val" style="

```

```

        <div class="card"><div class="lbl">Cash Available</div><div class="val" style=
        <div class="card"><div class="lbl">What-If Value</div><div class="val" style=
        <div class="card"><div class="lbl">Market Sentiment</div><div class="val" sty
    </div>

    <div class="sec"><h2>🌐 Market Mood</h2><div class="mood">{analysis.get('market
    <div class="sec"><h2>🗺️ Sector Heat Map</h2><p>🔥 <span class="hot"><b>Hot:</b>

    <div class="sec">
        <h2>📈 Today's Picks</h2>
        <table>
            <tr><th>Ticker</th><th>Category</th><th>Risk</th><th>Confidence</th><th>Bes
                {picks_html}
            </tr>
        </table>
    </div>

    <div class="sec"><h2>👁️ Watchlist</h2><p>{watchlist_links}</p></div>
    <div class="sec"><h2>📅 Earnings & Events</h2><p>{analysis.get('earnings_events
    <div class="sec"><h2>🎲 What-If Simulator</h2><p>${STARTING_CAPITAL} → <strong>

    <div class="sec">
        <h2>👛 Portfolio Positions</h2>
        <table>
            <tr><th>Ticker</th><th>Shares</th><th>Avg Price</th><th>Current Price</th><
                {positions_html}
            </tr>
        </table>
    </div>

    <p class="footer">⚠️ Educational purposes only. Not financial advice. Starting
</body>
</html>"""

```

```

        with open("dashboard.html", "w") as f:
            f.write(html)
        print(" ✅ Dashboard updated: dashboard.html")
    except Exception as e:
        print(f" ❌ Dashboard error: {e}")

# ┌────────────────────────────────────────────────────────────────────────────────┐
# │                                                                 MAIN RUNS                                                                 │
# └────────────────────────────────────────────────────────────────────────────────┘

def run_daily():
    if PAUSED:
        print("⏸️ Bot is paused. Set PAUSED = False in the CONFIG section to res
        return

```

```

print(f"\n{'='*60}")
print(f" Daily Stock Advisor - {datetime.now().strftime('%A, %B %d, %Y %H:')}
print(f"{'='*60}\n")

try:
    print(f" Fetching market data...")
    quotes = get_all_quotes()
    news = get_market_news()

    print(f" Checking volatility alerts...")
    alerts = check_volatility_alerts(quotes)
    if alerts:
        rows = "".join([f"<li><strong>{a['ticker']}</strong> {a['direction']}"
        send_email(f" Volatility Alert - {datetime.now().strftime('%B %d, %

    print(f" Checking commodity correlations...")
    commodity_data = get_commodity_correlation()

    print(f" Running multi-model consensus...")
    consensus = get_consensus_analysis(quotes, news)

    print(f" Running main AI analysis (this takes ~30 seconds)...")
    analysis = analyze_stocks(quotes, news)

    print(f" Calculating portfolio...")
    portfolio_value = get_portfolio_value(quotes)
    harvest_candidates = check_tax_loss_harvesting(quotes)
    what_if = what_if_simulator()
    dca_summary = get_dca_summary()

    print(f" Generating PDF report...")
    pdf_file = generate_pdf_report(analysis, portfolio_value, what_if, consen

    print(f" Generating audio summary...")
    audio_file = generate_audio_summary(analysis, portfolio_value)

    print(f" Sending daily email...")
    html = build_daily_email(analysis, quotes, portfolio_value, consensus, co
    send_email(
        subject = f" Daily Stock Picks - {datetime.now().strftime('%B %
        html_content= html,
        attachments = [f for f in [pdf_file, audio_file] if f]
    )

    print(f" Updating dashboard...")
    update_dashboard(analysis, portfolio_value, what_if, consensus, quotes)

```



```

print("📝 Saving to learning log...")
save_today_to_learning_log(analysis, quotes)
update_performance_in_log(quotes)

print(f"\n✅ Daily run complete!\n")

except Exception as e:
    print(f"❌ Error in daily run: {e}")
    send_email("⚠️ Stock Advisor Error", f"<h2>Error in daily run</h2><p>{str

def run_premarket():
    if PAUSED:
        return
    print("🔍 Running pre-market scan...")
    try:
        quotes = get_all_quotes()
        movers = sorted(quotes, key=lambda x: abs(x["change_percent"]), reverse=
        rows = "".join([f"<tr><td><strong>{q['ticker']}</strong></td><td>${q['
        html = f"<html><body style='font-family:Arial;max-width:600px;margin:0
        send_email(f"🔍 Pre-Market Movers - {datetime.now().strftime('%B %d, %Y')}
    except Exception as e:
        print(f"❌ Pre-market error: {e}")

def run_weekly():
    if PAUSED:
        return
    print("📊 Sending weekly report...")
    try:
        quotes = get_all_quotes()
        portfolio_value = get_portfolio_value(quotes)
        what_if = what_if_simulator()
        html = build_weekly_email(quotes, portfolio_value, what_if)
        send_email(f"📊 Weekly Stock Report - Week of {(datetime.now()-timedelta(
    except Exception as e:
        print(f"❌ Weekly report error: {e}")

def run_friday_alert():
    if PAUSED or datetime.now().weekday() != 4:
        return
    print("📅 Running Friday options alert...")
    try:
        quotes = get_all_quotes()
        volatile = [q for q in quotes if abs(q["change_percent"]) > 2]
        rows = "".join([f"<li><strong>{q['ticker']}</strong>: {'UP' if q['cha
        html = f"<html><body style='font-family:Arial;max-width:600px;margin:
        send_email(f"📅 Friday Options Expiration Alert", html)

```

#	ENTRY POINT
---	-------------

#	ENTRY POINT
---	-------------